



Semantic Fuzzing with Zest

Rohan Padhye
University of California, Berkeley
USA
rohanpadhye@cs.berkeley.edu

Caroline Lemieux
University of California, Berkeley
USA
clemieux@cs.berkeley.edu

Koushik Sen
University of California, Berkeley
USA
ksen@cs.berkeley.edu

Mike Papadakis
University of Luxembourg
Luxembourg
michail.papadakis@uni.lu

Yves Le Traon
University of Luxembourg
Luxembourg
yves.letraon@uni.lu

ABSTRACT

Programs expecting structured inputs often consist of both a *syntactic analysis stage*, which parses raw input, and a *semantic analysis stage*, which conducts checks on the parsed input and executes the core logic of the program. Generator-based testing tools in the lineage of QuickCheck are a promising way to generate random syntactically valid test inputs for these programs. We present *Zest*, a technique which automatically guides QuickCheck-like random-input generators to better explore the semantic analysis stage of test programs. *Zest* converts random-input generators into deterministic *parametric generators*. We present the key insight that mutations in the untyped parameter domain map to structural mutations in the input domain. *Zest* leverages program feedback in the form of code coverage and input validity to perform *feedback-directed parameter search*. We evaluate *Zest* against AFL and QuickCheck on five Java programs: Maven, Ant, BCEL, Closure, and Rhino. *Zest* covers $1.03\times$ – $2.81\times$ as many branches within the benchmarks’ semantic analysis stages as baseline techniques. Further, we find 10 new bugs in the semantic analysis stages of these benchmarks. *Zest* is the most effective technique in finding these bugs reliably and quickly, requiring at most 10 minutes on average to find each bug.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging.**

KEYWORDS

Structure-aware fuzzing, property-based testing, random testing

ACM Reference Format:

Rohan Padhye, Caroline Lemieux, Koushik Sen, Mike Papadakis, and Yves Le Traon. 2019. Semantic Fuzzing with Zest. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA ’19)*, July 15–19, 2019, Beijing, China. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3293882.3330576>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ISSTA ’19, July 15–19, 2019, Beijing, China

© 2019 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-6224-5/19/07...\$15.00

<https://doi.org/10.1145/3293882.3330576>

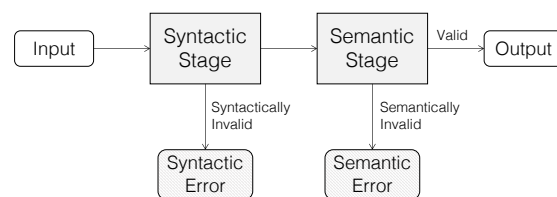


Figure 1: Inputs to a program taking structured inputs can be either syntactically or semantically invalid or just valid.

1 INTRODUCTION

Programs expecting complex structured inputs often process their inputs and convert them into suitable data structures before invoking the actual functionality of the program. For example, a build system such as Apache Maven first parses its input as an XML document and checks its conformance to a schema before invoking the actual build functionality. Document processors, Web browsers, compilers and various other programs follow this same check-then-run pattern.

In general, such programs have an input processing pipeline consisting of two stages: a syntax parser and a semantic analyzer. We illustrate this pipeline in Figure 1. The syntax parsing stage translates the raw input into an internal data structure that can be easily processed (e.g. an abstract syntax tree) by the rest of the program. The semantic analysis stage checks if an input satisfies certain semantic constraints (e.g. if an XML input fits a specific schema), and executes the core logic of the program. Inputs may be rejected by either stage if they are *syntactically* or *semantically invalid*. If an input passes both stages, we say the input is *valid*.

Automatically testing such programs is challenging. The difficulty lies in synthesizing inputs that (1) satisfy complex constraints on their structure and (2) exercise a variety of code paths in the semantic analysis stages and beyond. Random input generation is a popular technique for such scenarios because it can easily scale to execute a large number of test cases. Developers can write domain-specific *generators* from which random syntactically valid inputs—such as XML documents and abstract syntax trees—can be sampled. Popularized by QuickCheck [30], this approach has been adopted by many generator-based testing tools [16, 19, 32, 34, 38, 43, 49, 61]. QuickCheck-like test frameworks are now available in many programming languages such as Java [44], PHP [10], Python [11],

JavaScript [12], Scala [14], and Clojure [15]. Many commercial black-box fuzzing tools, such as Peach [13], beSTORM [7], Cyberflood [9], and Codenomicon [8], also leverage generators for network protocols or file formats. However, in order to effectively exercise the semantic analyses in the test program, the generators need to be tuned to produce inputs that are also *semantically* valid. For example, the developers of CSmith [72], a tool that generates random C programs for testing compilers, spent significant effort manually tuning their generator to reliably produce valid C programs and to maximize code coverage in the compilers under test.

In this paper, we present Zest, a technique for *automatically* guiding QuickCheck-like input generators to exercise various code paths in the semantic analysis stages of programs. Zest incorporates feedback from the test program in the form of *semantic validity* of test inputs and the *code coverage* achieved during test execution. The feedback is then used to generate new inputs via mutations.

Coverage-guided fuzzing (CGF) tools such as AFL [74] and libFuzzer [45] have gained a huge amount of popularity recently due to their effectiveness in finding critical bugs and security vulnerabilities in widely-used software systems. CGF works by randomly mutating known inputs via operations such as bit flips and byte-level splicing to produce new inputs. If the mutated inputs lead to new code coverage in the test program, they are saved for subsequent mutation. Of course, such mutations usually lead to invalid syntax. Naturally, most of the bugs found by these CGF tools lie in the syntax analysis stage of programs. CGF tools often require many hours or days of fuzzing to discover deeper bugs [47], which makes them impractical for use in continuous integration systems with limited testing time budgets.

Our proposed technique, Zest, adapts the algorithm used by CGF tools in order to quickly explore the *semantic analysis* stages of test programs. Zest first converts QuickCheck-like random-input generators into deterministic *parametric generators*, which map a sequence of untyped bits, called the “parameters”, to a syntactically valid input. The key insight in Zest is that *bit-level* mutations on these parameters correspond to *structural* mutations in the space of syntactically valid inputs. Zest then applies a CGF algorithm on the domain of parameters, in order to guide the test-input generation towards semantic validity and increased code coverage in the semantic analysis stages.

We have integrated Zest into the open-source JQF framework [60]: <https://github.com/rohanpadhye/jqf>. We evaluate Zest on five real-world Java benchmarks and compare it to AFL [74] and (a Java port of) QuickCheck [44]. Our results show that the Zest technique achieves significantly higher code coverage in the semantic analysis stage of each benchmark. Further, during our evaluation, we find 10 new bugs in the semantic analysis stages of our benchmarks. We find Zest to be the most effective technique for reliably and quickly triggering these *semantic bugs*. For each benchmark, Zest discovers an input triggering every semantic bug in at most 10 minutes on average. Zest complements AFL, which is best suited for finding syntactic bugs.

To summarize, we make the following contributions:

- We convert existing random-input generators into deterministic *parametric generators*, enabling a mapping from bit-level mutations of parameters to structural mutations of inputs.

- We present an algorithm that combines parametric generators with *feedback-directed parameter search*, in order to effectively explore the semantic analysis stages of programs.
- We evaluate our Java-based implementation of Zest against AFL and QuickCheck on five real-world benchmarks to compare their effectiveness in exercising code paths and discovering new bugs within the semantic analysis stage.

2 BACKGROUND

2.1 Generator-Based Testing

Generator-based testing tools [16, 30, 32, 34, 38, 43, 61, 72] allow users to write generator programs for producing inputs that belong to a specific type or format. These random-input generators are *non-deterministic*, i.e., they sample a new input each time they are executed. Figure 2 shows a generator for XML documents in the junit-quickcheck [44] framework, which is a Java port of QuickCheck [30]. When `generate()` is called, the generator uses the Java standard library XML DOM API to generate a random XML document. It constructs the root element of the document by invoking `genElement` (Line 4). Then, `genElement` uses repeated calls to methods of `random` to generate the element’s tag name (Line 9), any embedded text (Lines 19, 20, and in `genString`), and the number of children (Line 13); it recursively calls `genElement` to generate each child node. We omitted code to generate attributes, but it can be done analogously.

Figure 3 contains a sample test harness method `testProgram`, identified by the `@Property` annotation. This method expects a test input `xml` of type `XMLDocument`; the `@From` annotation indicates that inputs will be randomly generated using the `XMLGenerator.generate()` API. When invoked with a generated XML document, `testProgram` serializes the document (Line 3) and invokes the `readModel` method (Line 9), which parses an input string into a domain-specific model. For example, Apache Maven parses `pom.xml` files into an internal Project Object Model (POM). The model creation fails if the input XML document string does not meet certain syntactic and semantic requirements (Lines 11, 13). If the model creation is successful, the check at Line 4 succeeds and the test harness invokes the method `runModel` at Line 5 to test one of the core functionalities of the program under test.

An XML generator like the one shown in Figure 2 generates random syntactically valid XML inputs; therefore Line 11 in Figure 3 will never be executed. However, the generated inputs may not be *semantically* valid. That is, the inputs generated by the depicted XML generator do not necessarily conform to the schema expected by the application. In our example, the `readModel` method could throw a `ModelException` and cause the assumption at Line 4 to fail. If this happens, QuickCheck simply discards the test case and tries again. Writing generators that produce semantically valid inputs by construction is a challenging manual effort.

When we tested Apache Maven’s model reader for `pom.xml` files using a generator similar to Figure 2, we found that only 0.09% of the generated inputs were semantically valid. Moreover, even if the generator manages to generate semantically valid inputs, it may not generate inputs that exercise a variety of code paths in the semantic analysis stage. In our experiments with Maven, the QuickCheck approach covers less than one-third of the branches

```

1 class XMLGenerator implements Generator<XMLDocument> {
2   @Override // For Generator<XMLDocument>
3   public XMLDocument generate(Random random) {
4     XMLElement root = genElement(random, 1);
5     return new XMLDocument(root);
6   }
7   private XMLElement genElement(Random random, int depth) {
8     // Generate element with random name
9     String name = genString(random);
10    XMLElement node = new XMLElement(name);
11    if (depth < MAX_DEPTH) { // Ensures termination
12      // Randomly generate child nodes
13      int n = random.nextInt(MAX_CHILDREN);
14      for (int i = 0; i < n; i++) {
15        node.appendChild(genElement(random, depth+1));
16      }
17    }
18    // Maybe insert text inside element
19    if (random.nextBool()) {
20      node.addText(genString(random));
21    }
22    return node;
23  }
24  private String genString(Random random) {
25    // Randomly choose a length and characters
26    int len = random.nextInt(1, MAX_STRLEN);
27    String str = "";
28    for (int i = 0; i < len; i++) {
29      str += random.nextChar();
30    }
31    return str;
32  }
33 }

```

Figure 2: A simplified XML document generator.

```

1 @Property
2 void testProgram(@From(XMLGenerator.class) XMLDocument xml) {
3   Model model = readModel(xml.toString());
4   assume(model != null); // validity
5   assert(runModel(model) == success);
6 }
7 private Model readModel(String input) {
8   try {
9     return ModelReader.readModel(input);
10  } catch (XMLParseException e) {
11    return null; // syntax error
12  } catch (ModelException e) {
13    return null; // semantic error
14  }
15 }

```

Figure 3: A junit-quickcheck property that tests an XML-based component.

in the semantic analysis stage than our proposed technique does. Fundamentally, this is because of the lack of coupling between the generators and the program under test.

2.2 Coverage-Guided Fuzzing

Algorithm 1 describes coverage-guided fuzzing (CGF). CGF operates on raw test inputs represented as strings or byte-arrays. The algorithm maintains a set S of important test inputs, which are used as candidates for future mutations. S is initialized with a user-provided set of initial seed inputs I (Line 1). The algorithm repeatedly cycles through the elements of S (Line 5), each time picking an input from which to generate new inputs via mutation. The number of new inputs to generate in this round (Line 6) is determined by an implementation-specific heuristic. CGF generates new inputs by applying one or more random mutation operations

Algorithm 1 Coverage-guided fuzzing.

Input: program p , set of initial inputs I

Output: a set of test inputs and failing inputs

```

1:  $S \leftarrow I$ 
2:  $\mathcal{F} \leftarrow \emptyset$ 
3:  $totalCoverage \leftarrow \emptyset$ 
4: repeat
5:   for  $input$  in  $S$  do
6:     for  $1 \leq i \leq \text{NUMCANDIDATES}(input)$  do
7:        $candidate \leftarrow \text{MUTATE}(input, S)$ 
8:        $coverage, result \leftarrow \text{RUN}(p, candidate)$ 
9:       if  $result = \text{FAILURE}$  then
10:         $\mathcal{F} \leftarrow \mathcal{F} \cup candidate$ 
11:       else if  $coverage \not\subseteq totalCoverage$  then
12:         $S \leftarrow S \cup \{candidate\}$ 
13:         $totalCoverage \leftarrow totalCoverage \cup coverage$ 
14: until given time budget expires
15: return  $S, \mathcal{F}$ 

```

on the base input (Line 7). These mutations may include operations that combine subsets of other inputs in S . The given program is then executed with each newly generated input (Line 8). The result of a test execution can either be SUCCESS or FAILURE. If an input causes a test failure, it is added to the failure set $\mathcal{F}$ (Line 10).

The key to the CGF algorithm is that instead of treating the test program as a black-box as QuickCheck does, it instruments the test program to provide dynamic feedback in the form of code coverage for each run. The algorithm maintains in the variable *totalCoverage* the set of all *coverage points* (e.g. program branches) covered by the existing inputs. If the successful execution of a generated input leads to the discovery of new coverage points (Line 11), then this input is added to the set S for subsequent fuzzing (Line 12) and the newly covered coverage points are added to *totalCoverage*. (Line 13).

The whole process repeats until a time budget expires. Finally, CGF returns the generated corpus of test inputs S and failing inputs $\mathcal{F}$ (Line 15). CGF can either be used as a technique to discover inputs that expose bugs—in the form of crashes or assertion violations—or to automatically generate a corpus of test inputs that cover various program features.

A key limitation of existing CGF tools is that they work without any knowledge about the syntax of the input. State-of-the-art CGF tools [25, 28, 45, 50, 63, 74] treat program inputs as sequences of bytes. This choice of representation also influences the design of their mutation operations, which include bit-flips, arithmetic operations on word-sized segments, setting random bytes to random or “interesting” values (e.g. 0, MAX_INT), etc. These mutations are tailored towards exercising various code paths in programs that parse inputs with a compact syntax, such as parsers for media file formats, decompression routines, and network packet analyzers. CGF tools have been very successful in finding memory-corruption bugs (such as buffer overflows) in the syntax analysis stage of such programs due to incorrect handling of unexpected inputs.

Unfortunately, this approach often fails to exercise the core functions of software that expects highly structured inputs. For example, when AFL [74] is applied on a program that processes XML input data, a typical input that it saves looks like:

```
<a b>ac&#84;a>
```

which exercises code paths that deal with syntax errors. In this case, an error-handling routine for unmatched start and end XML tags. It is very difficult to generate inputs that will exercise new, interesting code paths in the semantic analysis stage of a program via these low-level mutations. Often, it is necessary to run CGF tools for hours or days on end in order to find non-trivial bugs, making them impractical for use in a continuous integration setting.

3 PROPOSED TECHNIQUE

Our approach, Zest, adds the power of coverage-guided fuzzing to generator-based testing. First, Zest converts a random-input generator into an equivalent deterministic *parametric generator*. Zest then uses *feedback-directed parameter search* to search through the parameter space. This technique augments the CGF algorithm by keeping track of code coverage achieved by valid inputs. This enables it to guide the search towards deeper code paths in the semantic analysis stage.

3.1 Parametric Generators

Before defining parametric generators, let us return to the random XML generator from Figure 2. Let us consider a particular path through this generator, concentrating on the calls to `nextInt`, `nextBool`, and `nextChar`. The following sequence of calls will be our running example (some calls omitted for space):

Call → result	Context
<code>random.nextInt(1, MAX_STRLen) → 3</code>	Root: name length (Line 26)
<code>random.nextChar() → 'f'</code>	Root: name[0] (Line 29)
<code>random.nextChar() → 'o'</code>	Root: name[1] (Line 29)
<code>random.nextChar() → 'o'</code>	Root: name[2] (Line 29)
<code>random.nextInt(MAX_CHILDREN) → 2</code>	Root: # children (Line 13)
<code>random.nextInt(1, MAX_STRLen) → 3</code>	Child 1: name length (Line 26)
	⋮
<code>random.nextBool() → False</code>	Child 2: embed text? (Line 19)
<code>random.nextBool() → False</code>	Root: embed text? (Line 19)

The XML document produced when the generator makes this sequence of calls looks like:

$$x_1 = \langle \text{foo} \rangle \langle \text{bar} \rangle \text{Hello} \langle \text{bar} \rangle \langle \text{baz} \rangle / \rangle \langle \text{foo} \rangle.$$

In order to produce random typed values, the implementations of `random.nextInt`, `random.nextChar`, and `random.nextBool` rely on a pseudo-random source of *untyped* bits. We call these untyped bits “*parameters*”. The parameter sequence for the example above, annotated with the calls which consume the parameters, is:

$$\sigma_1 = \underbrace{0000\ 0010}_{\text{nextInt}(1, \dots) \rightarrow 3} \underbrace{0110\ 0110}_{\text{nextChar}() \rightarrow 'f'} \dots \underbrace{0000\ 0000}_{\text{nextBool}() \rightarrow \text{False}}.$$

For example, here the function `random.nextInt(a, b)` consumes eight bit parameters as a byte, n , and returns $n \% (b - a) + a$ as a typed integer. For simplicity of presentation, we show each `random.nextXYZ` function consuming the same number of parameters, but they can consume different numbers of parameters.

We can now define a *parametric generator*. A parametric generator is a function that takes a sequence of untyped parameters such as σ_1 —the *parameter sequence*—and produces a structured input, such as the XML x_1 . A parametric generator can be implemented

by simply replacing the underlying implementation of `Random` to consult not a pseudo-random source of bits but instead a fixed sequence of bits provided as the parameters.

While this is a very simple change, making generators deterministic and explicitly dependent on a fixed parameter sequence enables us to make the following two key observations:

- (1) *Every untyped parameter sequence corresponds to a syntactically valid input*—assuming the generator only produces syntactically valid inputs.
- (2) *Bit-level mutations on untyped parameter sequences correspond to high-level structural mutations in the space of syntactically valid inputs.*

Observation (1) is true by construction. The `random.nextXYZ` functions are implemented to produce correctly-typed values no matter what bits the pseudo-random source—or in our case, the parameters—provide. Every sequence of untyped parameter bits correspond to some execution path through the generator, and therefore every parameter sequence maps to a syntactically valid input. We describe how we handle parameter sequences that are longer or shorter than expected with the example sequences σ_3 and σ_4 , respectively, below.

To illustrate observation (2), consider the following parameter sequence, σ_2 , produced by mutating just a few bits of σ_1 :

$$\sigma_2 = 0000\ 0010\ \underbrace{0101\ 0111}_{\text{nextChar}() \rightarrow 'W'} \dots 0000\ 0000.$$

As indicated by the annotation, all this parameter-sequence mutation does is change the value returned by the second call to `random.nextChar()` in our running example from ‘f’ to ‘W’. So the generator produces the following test-input:

$$x_2 = \langle \text{Woo} \rangle \langle \text{bar} \rangle \text{Hello} \langle \text{bar} \rangle \langle \text{baz} \rangle / \rangle \langle \text{Woo} \rangle.$$

Notice that this generated input is still syntactically valid, with “Woo” appearing both in the start and end tag delimiters. This is because the XML generator uses an internal DOM tree representation that is only serialized after the entire tree is generated. We get this syntactic-validity-preserving structural mutation for free, *by construction*, and without modifying the underlying generators.

Mutating the parameter sequence can also result in more drastic high-level mutations. Suppose that σ_1 is mutated to influence the first call to `random.nextInt(MAX_CHILDREN)` as follows:

$$\sigma_3 = 0000 \dots \underbrace{0000\ 0001}_{\text{nextInt}(MAX_CHILDREN) \rightarrow 1} \dots 0000.$$

Then the root node in the generated input will have only one child:

$$x_3 = \langle \text{foo} \rangle \langle \text{bar} \rangle \text{Hello} \langle \text{bar} \rangle \blacksquare \langle \text{foo} \rangle$$

($\blacksquare$ designates the absence of `<baz />`). Since the remaining values in the untyped parameter sequence are the same, the first child node in x_3 —`<bar>Hello</bar>`—is identical to the one in x_1 . The parametric generator thus enables a structured mutation in the DOM tree, such as deleting a sub-tree, by simply changing a few values in the parameter sequence. Note that this change results in fewer `random.nextXYZ` calls by the generator; the unused parameters in the tail of the parameter sequence will simply be ignored by the parametric generator.

Algorithm 2 The Zest algorithm, pairing parametric generators with feedback-directed parameter search. Additions to Algorithm 1 highlighted in grey.

Input: program p , generator q

Output: a set of test inputs and failing inputs

```

1:  $S \leftarrow \{\text{RANDOM}\}$ 
2:  $\mathcal{F} \leftarrow \emptyset$ 
3:  $\text{totalCoverage} \leftarrow \emptyset$ 
4:  $\text{validCoverage} \leftarrow \emptyset$ 
5:  $g \leftarrow \text{MAKEPARAMETRIC}(q)$ 
6: repeat
7:   for  $\text{param}$  in  $S$  do
8:     for  $1 \leq i \leq \text{NUMCANDIDATES}(\text{param})$  do
9:        $\text{candidate} \leftarrow \text{MUTATE}(\text{param}, S)$ 
10:       $\text{input} \leftarrow g(\text{candidate})$ 
11:       $\text{coverage}, \text{result} \leftarrow \text{RUN}(p, \text{input})$ 
12:      if  $\text{result} = \text{FAILURE}$  then
13:         $\mathcal{F} \leftarrow \mathcal{F} \cup \text{candidate}$ 
14:      else
15:        if  $\text{coverage} \not\subseteq \text{totalCoverage}$  then
16:           $S \leftarrow S \cup \{\text{candidate}\}$ 
17:           $\text{totalCoverage} \leftarrow \text{totalCoverage} \cup \text{coverage}$ 
18:        if  $\text{result} = \text{VALID}$  and  $\text{coverage} \not\subseteq \text{validCoverage}$  then
19:           $S \leftarrow S \cup \{\text{candidate}\}$ 
20:           $\text{validCoverage} \leftarrow \text{validCoverage} \cup \text{coverage}$ 
21: until given time budget expires
22: return  $g(S), g(\mathcal{F})$ 

```

For our final example, suppose σ_1 is mutated as follows:

$$\sigma_4 = 0000 \ 0011 \ \dots \ \underbrace{0000 \ 0001}_{\text{nextBool}() \rightarrow \text{True}} \ \underbrace{0000 \ 0000}_{\text{nextInt}(1, \dots) \rightarrow 1}.$$

Notice that after this mutation, the last 8 parameters are consumed by nextInt instead of by nextBool (ref. σ_1). But, note that nextInt still returns a valid typed value even though the parameters were originally consumed by nextBool.

At the input level, this modifies the call sequence so that the decision to embed text in the second child of the document becomes True. Then, the last parameters are used by nextInt to choose an embedded text length of 1 character. However, one problem remains: to generate the content of the embedded text, the generator needs more parameter values than σ_4 contains. In Zest, we deal with this by appending pseudo-random values to the end of the parameter sequence on demand. We use a fixed random seed to maintain determinism. For example, suppose the sequence is extended as:

$$\sigma'_4 = 0000 \ \dots \ 0001 \ 0000 \ 0000 \ \underbrace{0100 \ 1100}_{\text{nextChar}() \rightarrow \text{'H'}} \ \underbrace{0000 \ 0000}_{\text{nextBool}() \rightarrow \text{False}}$$

Then the parametric generator would produce the test-input:

$$x_4 = \langle \text{foo} \rangle \langle \text{bar} \rangle \text{Hello} \langle \text{bar} \rangle \langle \text{baz} \rangle \text{H} \langle \text{baz} \rangle \langle \text{foo} \rangle.$$

3.2 Feedback-Directed Parameter Search

Algorithm 2 shows the Zest algorithm, which guides parametric generators to produce inputs that get deeper into the semantic

analysis stage of programs using *feedback-directed parameter search*. The Zest algorithm resembles Algorithm 1, but acts on parameter sequences rather than the raw inputs to the program. It also extends the CGF algorithm by keeping track of the coverage achieved by *semantically valid inputs*. We highlight the differences between Algorithms 2 and 1 in grey.

Like Algorithm 1, Zest is provided a program under test p . Unlike Algorithm 1 which assumes seed inputs, the set of parameter sequences is initialized with a random sequence (Line 1). Additionally, Zest is provided a generator g , which it automatically converts to a parametric generator g (Line 5). In an abuse of notation, we use $g(S)$ to designate the set of inputs generated by running g over the parameter sequences in S , i.e. $g(S) = \{g(s) : s \in S\}$.

Along with totalCoverage , which maintains the set of coverage points in p covered by *all* inputs in $g(S)$, Zest also maintains validCoverage , the set of coverage points covered by the (semantically) valid inputs in $g(S)$. This is initialized at Line 4.

New parameter sequences are generated using standard CGF mutations at Line 9; see Section 4 for details. New inputs are generated by running the sequences through the parametric generator (Line 10). The program p is then executed on each input. During the execution, in addition to code-coverage and failure feedback, the algorithm records in the variable result whether the input is valid or not. In particular, result can be any of $\{\text{VALID}, \text{INVALID}, \text{FAILURE}\}$. An input is considered invalid if it leads to a violation of any assumption in the test harness (e.g. Figure 3 at Line 4), which is how we capture application-specific semantic validity.

As in Algorithm 1, a newly generated parameter sequence is added to the set S at Lines 15–17 of Algorithm 2 if the corresponding input produces new code coverage. Further, if the corresponding input is *valid* and covers a coverage point that has not been exercised by *any previous valid input*, then the parameter sequence is added S and the cumulative valid coverage variable validCoverage is updated at Lines 18–20. Adding the parameter sequence to S under this new condition ensures that Zest keeps mutating valid inputs that exercise core program functionality. We hypothesize that this biases the search towards generating even more valid inputs and in turn increases code coverage in the semantic analysis stage.

As in Algorithm 1, the testing loop repeats until a time budget expires. Finally, Zest returns the corpus of generated test inputs $g(S)$ and failing inputs $g(\mathcal{F})$.

4 IMPLEMENTATION

Zest is implemented on top of the open-source JQF platform [60], which provides a framework for specifying algorithms for feedback-directed fuzz testing of Java programs. JQF dynamically instruments Java classes in the program under test using the ASM bytecode-manipulation framework [58] via a `javaagent`. The instrumentation allows JQF to observe code coverage events, e.g. the execution of program branches and invocation of virtual method calls.

Fuzzing “guidances” can plug into JQF to provide inputs and register callbacks for listening to code coverage events. JQF originally shipped with `AFLGuidance` and `NoGuidance`, which we use in our evaluation in Section 5. `AFLGuidance` uses a proxy program to exchange program inputs and coverage feedback with the external AFL tool; the overhead of this inter-process communication

is a negligible fraction of the test execution time. NoGuidance randomly samples inputs from `junit-quickcheck` [44] generators without using coverage feedback. We implement ZestGuidance in JQF, which biases these generators using Algorithm 2.

`junit-quickcheck` provides a high-level API for making random choices in the generators, such as generating random integers, time durations, and selecting random items from a collection. All of these methods indirectly rely on the underlying JDK method `java.util.Random.next(int bits)`, which returns bits from a pseudo-random stream. Zest replaces this pseudo-random stream with stored parameter sequences, which are extended on-demand.

Since `java.util.Random` polls byte-sized chunks from its underlying stream of pseudo-random bits, Zest performs mutations on the parameter sequences (Algorithm 2, Line 9) at the *byte-level*. The basic mutation procedure is as follows: (1) choose a random number m of mutations to perform sequentially on the original sequence, (2) for each mutation, choose a random length ℓ of bytes to mutate and an offset k at which to perform the mutation, and (3) replace the bytes from positions $[k, k + \ell)$ with ℓ randomly chosen bytes. The random numbers m and ℓ are chosen from a geometric distribution, which mostly provides small values without imposing an upper bound. We set the mean of this distribution to 4, since 4-byte ints are the most commonly requested random value.

5 EVALUATION

We evaluate Zest by measuring its effectiveness in testing the semantic analysis stages of five benchmark programs. We compare Zest with two baseline techniques: AFL and `junit-quickcheck` (referred to as simply QuickCheck hereon). AFL is known to excel in exercising the syntax analysis stage via coverage-guided fuzzing of raw input strings. We use version 2.52b, with “FidgetyAFL” configuration, which was found to match the performance of AFLFast [75]. QuickCheck uses the same generators as Zest but only performs random sampling without any feedback from the programs under test. Specifically, we evaluate the three techniques on two fronts: (1) the amount of code coverage achieved in the semantic analysis stage after a fixed amount of time, and (2) their effectiveness in triggering bugs in the semantic analysis stage.

Benchmarks. We use the following five real-world Java benchmarks as test programs for our evaluation:

- (1) Apache Maven [3] (99k LoC): The test reads a `pom.xml` file and converts it into an internal `Model` structure. The test driver is similar to the one in Figure 3. An input is valid if it is a valid XML document conforming to the POM schema.
- (2) Apache Ant [1] (223k LoC): Similar to Maven, this test reads a `build.xml` file and populates a `Project` object. An input is considered valid if it is a valid XML document and if it conforms to the schema expected by Ant.
- (3) Google Closure [4] (247k LoC) statically optimizes JavaScript code. The test driver invokes the `Compiler.compile()` on the input with the `SIMPLE_OPTIMIZATIONS` flag, which enables constant folding, function inlining, dead-code removal, etc.. An input is valid if Closure returns without error.
- (4) Mozilla Rhino [5] (89k LoC) compiles JavaScript to Java bytecode. The test driver invokes `Context.compileString()`. An input is valid if Rhino returns a compiled script.
- (5) Apache’s Bytecode Engineering Library (BCEL) [2] (61k LoC) provides an API to parse, verify and manipulate Java bytecode. Our test driver takes as input a `.class` file and uses the `Verifier` API to perform 3-pass verification of the class file according to the Java 8 specification [53]. An input is valid if BCEL finds no errors up to Pass 3A verification.

Experimental Setup. We make the following design decisions:

- **Duration:** We run each test-generation experiment for 3 hours. Researchers have used various timeouts to evaluate random test generation tools, from 2 minutes [37, 59] to 24 hours [25, 47]. We chose 3 hours as a middle ground. Our experiments justify this choice, as we found that semantic coverage plateaued after 2 hours in almost all experiments. Specifically, the number of semantic branches covered by Zest increased by less than 1% in the last hour of the runs.
- **Repetitions:** Due to the non-deterministic nature of random testing, the results may vary across multiple repetitions of each experiment. We therefore run each experiment 20 times and report statistics across the 20 repetitions.
- **Seeds and Dictionaries:** To bootstrap AFL, we need to provide some initial seed inputs. There is no single best strategy for selecting initial seeds [64]. Researchers have found success using varying strategies ranging from large seed corpora to single empty files [47]. In our evaluation, we provide AFL one valid seed input per benchmark that covers various domain-specific semantic features. For example, in the Closure and Rhino benchmarks, we use the entire ReactJS library [6] as a seed. We also provide AFL with *dictionaries* of benchmark-specific strings (e.g. keywords, tag names) to inject into inputs during mutation. The generator-based tools Zest and QuickCheck do not rely on meaningful seeds.
- **Generators:** Zest and QuickCheck use hand-written input generators. For Maven and Ant, we use an XML document generator similar to Figure 2, of around 150 lines of Java code. It generates strings for tags and attributes by randomly choosing strings from a list of string literals scraped from class files in Maven and Ant. For Closure and Rhino, we use a generator for a subset of JavaScript that contains about 300 lines of Java code. The generator produces strings that are syntactically valid JavaScript programs. Finally, the BCEL generator (~500 LoC) uses the BCEL API to generate `JavaClass` objects with randomly generated fields, attributes and methods with randomly generated bytecode instructions. All generators were written by one of the authors of this paper in less than two hours each. Although these generators produce syntactically valid inputs, no effort was made to produce semantically valid inputs; doing so can be a complex and tedious task [72].

The generators, seeds, and dictionaries have been made publicly available at <https://goo.gl/GfLRzA>. All experiments are conducted on a machine with Intel(R) Core(TM) i7-5930K 3.50GHz CPU and 16GB of RAM running Ubuntu 18.04.

Syntax and Semantic Analysis Stages in Benchmarks. Zest is specifically designed to exercise the semantic analysis stages of

Table 1: Description of benchmarks with prefixes of class/package names corresponding to syntactic and semantic analyses.

Name	Version	Syntax Analysis Classes	Semantic Analysis Classes
Maven	3.5.2	org/codehaus/plexus/util/xml	org/apache/maven/model
Ant	1.10.2	com/sun/org/apache/xerces	org/apache/tools/ant
Closure	v20180204	com/google/javascript/jscomp/parsing	com/google/javascript/jscomp/[A-Z]
Rhino	1.7.8	org/mozilla/javascript/Parser	org/mozilla/javascript/(optimizer CodeGenerator)
BCEL	6.2	org/apache/bcel/classfile	org/apache/bcel/verifier

programs. To evaluate Zest’s effectiveness in this regard, we manually identify the components of our benchmark programs which correspond to syntax and semantic analysis stages. Table 1 lists prefix patterns that we match on the fully-qualified names of classes in our benchmarks to classify them in either stage. Section 5.1 evaluates the code coverage achieved within the classes identified as belonging to the semantic analysis stage. Section 5.2 evaluates the bug-finding capabilities of each technique for bugs that arise in the semantic analysis classes. Section 6 discusses some findings in the *syntax* analysis classes, whose testing is outside the scope of Zest.

5.1 Coverage of Semantic Analysis Classes

Instead of relying on our own instrumentation, we use a third party tool, the widely used Eclemma-JaCoCo [42] library, for measuring code coverage in our Java benchmarks. Specifically, we measure *branch coverage* within the semantic analysis classes from Table 1; we refer to these branches as *semantic branches* for short.

To approximate the coverage of the semantic branches covered via the selected test drivers, we report the percentage of total semantic branches covered. Note, however, that this is a *conservative*, i.e. low, estimate. This is because the total number of semantic branches includes some branches not reachable from the test driver. We make this approximation as it is not feasible to statically determine the number of branches reachable from a given entry point, especially in the presence of virtual method dispatch. We expect the percent of semantic branches reachable from our test drivers to be much lower than 100%; therefore, the relative differences between coverage are more important than the absolute percentages.

Figure 4 plots the semantic branch coverage achieved by each of Zest, AFL, and QuickCheck on the five benchmark programs across the 3-hour-long runs. In the plots, solid lines designate means and shaded areas designate 95% confidence intervals across the 20 repetitions. Interestingly, the variance in coverage is quite low for all techniques except QuickCheck. Since AFL is initialized with valid seed inputs, its initial coverage is non-zero; nonetheless, it is quickly overtaken by Zest, usually within the first 5 minutes.

Zest significantly outperforms baseline techniques in exercising branches within the semantic analysis stage, achieving statistically significant increases for all benchmarks. Zest covers as much as $2.81\times$ as many semantic branches covered by the best baseline technique for Maven (Figure 4a). When looking at our Javascript benchmarks, we see that Zest’s advantage over QuickCheck is more slight in Rhino (Figure 4b) than in Closure (Figure 4c). This may be because Closure, which performs a variety of static code optimizations on JavaScript programs, has many input-dependent paths. Rhino, on the other hand, directly compiles valid JavaScript

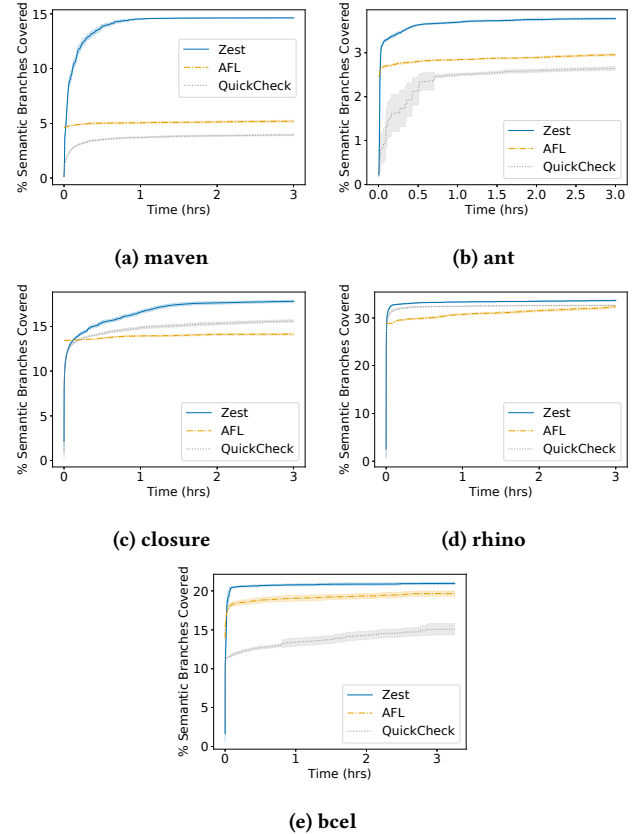


Figure 4: Percent coverage of *all* branches in semantic analysis stage of the benchmark programs. Lines designate means and shaded regions 95% confidence intervals.

to JVM bytecode, and thus has fewer input-dependent paths for Zest to discover through feedback-driven parameter search.

Note that in some benchmarks AFL has an edge in coverage over QuickCheck (Figure 4a, 4b, 4e), and vice-versa (Figure 4c, 4d). For BCEL, this may be because the input format is a compact syntax, on which AFL generally excels. The difference between the XML and JavaScript benchmarks may be related to the ability of randomly-sampled inputs from the generator to achieve broad coverage. It is much more likely for a random syntactically valid JavaScript program to be semantically valid than a random syntactically valid XML document to be a valid POM file, for example. The fact that

Zest dominates the baseline approaches in all these cases suggests that it is more robust to generator quality than QuickCheck.

5.2 Bugs in the Semantic Analysis Classes

Each of Zest, AFL, and QuickCheck keep track of generated inputs which cause test failures. Ideally, for any given input, the test program should either process it successfully or reject the input as invalid using a documented mechanism, such as throwing a checked `ParseException` on syntax errors. Test *failures* correspond either to assertion violations or to undocumented runtime exceptions being thrown during test execution, such as a `NullPointerException`. Test failures can occur during the processing of either valid or invalid inputs; the latter can lead to failures within the syntax or semantic analysis stages themselves.

Across all our experiments, the various fuzzing techniques generated over 95,000 failing inputs that correspond to over 3,000 unique stack traces. We manually triaged these failures by filtering them based on exception type, message text, and source location, resulting in a corpus of what we believe are 20 unique bugs. We have reported each of these bugs to the project developers. At the time of writing: 5 bugs have been fixed, 10 await patches, and 5 reports have received no response.

We classify each bug as *syntactic* or *semantic*, depending on whether the corresponding exception was raised within the syntactic or semantic analysis classes, respectively (ref. Table 1). Of the 20 unique bugs we found, 10 were syntactic and 10 were semantic.

Here, we evaluate Zest in discovering *semantic bugs*, for which it is specifically designed. Section 6 discusses the syntactic bugs we found, whose discovery was not Zest's goal.

Table 2 enumerates the 10 semantic bugs that we found across four of the five benchmark programs. The bugs have been given unique IDs—represented as circled letters—for ease of discussion. The table also lists the type of exception thrown for each bug. To evaluate the effectiveness of each of the three techniques in discovering these bugs, we use two metrics. First, we are interested in knowing whether a given technique reliably finds the bug across repeated experiments. We define *reliability* as the percentage of the 20 runs (of 3-hours each) in which a given technique finds a particular bug at least once. Second, we measure the *mean time to find* (MTF) the first input that triggers the given bug, across the repetitions in which it was found. Naturally, a shorter MTF is desirable. For each bug, we circle the name of the technique that is the most effective in finding that bug. We define *most effective* as the technique with either the highest reliability, or if there is a tie in reliability, then the shortest MTF.

The table indicates that Zest is the most effective technique in finding 8 of the 10 bugs; in the remaining two cases (F and O), Zest still finds the bugs with 100% reliability and in less than 20 seconds on average. In fact, Zest finds all the 10 semantic bugs in *at most 10 minutes on average*; 7 are found within the first 2 minutes on average. In contrast, AFL requires more than one hour to find 3 of the bugs (B, C, G), and simply does not find 5 of the bugs within the 3-hour time limit. This makes sense because AFL's mutations on the raw input strings do not guarantee syntactic validity; it generates much fewer inputs that reach the semantic analysis stage. QuickCheck discovers 8 of the 10 semantic bugs, but since it relies

on random sampling alone, its reliability is often low. For example, QuickCheck discovers B only 10% of the time, and N only 5% of the time; Zest discovers them 100% and 95% of the time, respectively. Overall, Zest is clearly the most effective technique in discovering bugs in the semantic analysis classes of our benchmarks.

Case studies. In Ant, B is triggered when the input `build.xml` document contains both an `<augment>` element and a `<target>` element inside the root `<project>` element, but when the `<augment>` element is missing an `id` attribute. This incomplete semantic check leads to an `IllegalStateException` for a component down the pipeline which tries to configure an Ant task. Following our bug report, this issue has been fixed starting Ant version 1.10.6.

In Rhino, I is triggered by a semantically valid input. Rhino successfully validates the input JavaScript program and then compiles it to Java bytecode. However, the compiled bytecode is corrupted, which results in a `VerifyError` being generated by the JVM. AFL does not find this bug at all. The Rhino developers confirmed the bug, though a fix is still pending.

In Closure, C is an NPE that is triggered in its dead-code elimination pass when handling arrow functions that reference undeclared variables, such as `"x => y"`. The generator-based techniques always find this bug and within just 8.8 seconds on average, while AFL requires more than 90 minutes and only finds it in 20% of the runs. The Closure developers fixed this issue after our report.

D is a bug in Closure's semantic analysis of variable declarations. The bug is triggered when a new variable is declared after a `break` statement. Although everything immediately after a `break` statement is unreachable code, variable declarations in JavaScript are hoisted and therefore cannot be removed. Zest is the only technique that discovered this bug. A sample input Zest generated is:

```
while ((l_0)){
  while ((l_0)){
    if ((l_0)) { break;var l_0;continue }
    { break;var l_0 }
  }
}
```

U was the most elusive bug that we encountered. Zest is the only technique that finds it and it does so in only one of the 20 runs. An exception is triggered by the following input:

```
((o_0) => (((o_0) *= (o_0))
  < ((i_1) &= ((o_0)((undefined)[((i_1, o_0, a_2) => {
    if ((i_1)) { throw ((false).o_0) }
  }))(y_3))))((new (null)((true))))))
```

The issue is perhaps rooted in Closure's attempt to compile-time evaluate `undefined[undefined](...)`. The developers acknowledged the bug but have not yet published a fix. These complex examples demonstrate both the power of Zest's generators, which reduce the search space to syntactically valid inputs, as well as the effectiveness of its feedback-directed parameter search.

6 DISCUSSION AND THREATS TO VALIDITY

Zest and QuickCheck make use of generators for synthesizing inputs that are syntactically valid by construction. By design, these tools do not exercise code paths corresponding to parse errors in the syntax analysis stage. In contrast, AFL performs mutations directly on raw input strings. Byte-level mutations on raw inputs usually

Table 2: The 10 new bugs found in the semantic analysis stages of benchmark programs. The tools Zest, AFL, and QuickCheck (QC) are evaluated on the *mean time to find* (MTF) each bug across the 20 repeated experiments of 3 hours each as well as the *reliability* of this discovery, which is the percentage of the 20 repetitions in which the bug was triggered at least once. For each bug, the highlighted tool is statistically significantly more effective at finding the bug than unhighlighted tools.

Bug ID	Exception	Tool	Mean Time to Find (shorter is better)	Reliability
ant (B)	IllegalStateException	Zest	1 (99.45 sec)	100%
		AFL	6369.5 (6369.5 sec)	10%
		QC	1208.0 (1208.0 sec)	10%
closure (C)	NullPointerException	Zest	1 (8.8 sec)	100%
		AFL	5496.25 (5496.25 sec)	20%
		QC	1 (8.8 sec)	100%
closure (D)	RuntimeException	Zest	460.42 (460.42 sec)	60%
		AFL	✗	0%
		QC	✗	0%
closure (U)	IllegalStateException	Zest	534.0 (534.0 sec)	5%
		AFL	✗	0%
		QC	✗	0%
rhino (G)	IllegalStateException	Zest	1 (8.25 sec)	100%
		AFL	5343.0 (5343.0 sec)	20%
		QC	1 (9.65 sec)	100%
rhino (F)	NullPointerException	Zest	1 (18.6 sec)	100%
		AFL	✗	0%
		QC	1 (9.85 sec)	100%
rhino (H)	ClassCastException	Zest	245.18 (245.18 sec)	85%
		AFL	✗	0%
		QC	362.43 (362.43 sec)	35%
rhino (J)	VerifyError	Zest	1 (94.75 sec)	100%
		AFL	✗	0%
		QC	229.5 (229.5 sec)	80%
bcel (O)	ClassFormatException	Zest	1 (19.5 sec)	100%
		AFL	1 (5.85 sec)	100%
		QC	142.1 (142.1 sec)	100%
bcel (N)	AssertionViolatedException	Zest	1 (19.32 sec)	95%
		AFL	1082.22 (1082.22 sec)	90%
		QC	1 (15.0 sec)	5%

lead to inputs that do not parse. Consequently, AFL spends most of its time testing error paths within the syntax analysis stages.

In our experiments, AFL achieved the highest coverage within the syntax analysis classes of our benchmarks (ref. Table 1), 1.1×–1.6× higher than Zest’s syntax analysis coverage. Further, AFL discovered 10 syntactic bugs in addition to the bugs enumerated in Table 1: 3 in Maven, 6 in BCEL, and 1 in Rhino. These bugs were triggered by syntactically invalid inputs, which the generator-based tools do not produce. Zest does not attempt to target these bugs; rather, it is complementary to AFL-like tools.

Zest assumes the availability of QuickCheck-like generators to exercise the semantic analysis classes and to find semantic bugs. Although this is no doubt an additional cost, the effort required to develop a structured-input generator is usually no more than the effort required to write unit tests with hand-crafted structured

inputs, which is usually an accepted cost. In fact, due to the growing popularity of generator-based testing tools like Hypothesis [11], ScalaCheck [14], PropEr [61], etc. a large number of off-the-shelf or automatically synthesized type-based generators are available. The Zest technique can, in principle, work with any such generator. When given a generator, Zest excels at exercising semantic analyses and is very effective in discovering semantic bugs.

We did not evaluate how Zest’s effectiveness might vary depending on the quality of generators, since we hand-wrote the simplest generators possible for our benchmarks. However, our results suggest that Zest’s ability to guide generation towards paths deep in the semantic analysis stage make its performance less tied to generator quality than pure random sampling as in QuickCheck.

The effectiveness of CGF tools like AFL is usually sensitive to the choice of seed inputs [47]. Although the relative differences

between the performance of Zest and AFL will likely vary with this choice, the purpose of our evaluation was to demonstrate that focusing on feedback-directed search in the space of syntactically valid inputs is advantageous. No matter what seed inputs one provides to conventional fuzzing tools, the byte-level mutations on raw inputs will lead to an enormous number of syntax errors. We believe that approaches like Zest complement CGF tools in testing different components of programs.

7 RELATED WORK

A lot of research has gone into automated test-input generation techniques, as surveyed by Anand et al. [17].

Randoop [59] and EvoSuite [36] generate JUnit tests for a particular class by incrementally trying and combining sequences of calls. During the generation of sequence of calls, both Randoop and EvoSuite take some form of feedback into account. These tools produce unit tests by directly invoking methods on the component classes. In contrast, Zest addresses the problem of generating *inputs* when given a test driver and an input generator.

UDITA [38] allows developers to write random-input generators in a QuickCheck-like language. UDITA then performs *bounded-exhaustive* enumeration of the paths through the generators, along with several optimizations. In contrast, Zest relies on random mutations over the entire parameter space but uses code coverage and input-validity feedback to guide the parameter search. It would be interesting to see if UDITA's search strategy could be augmented to selectively adjust bounds using code coverage and validity feedback; however, we leave this investigation as future work.

Targeted property-testing [54, 55] guides input generators used in property testing towards a user-specified fitness value using techniques such as hill climbing and simulated annealing. Gödel-Test [35] attempts to satisfy user-specified properties on inputs. It performs a meta-heuristic search for stochastic models that are used to sample random inputs from a generator. Unlike these techniques, Zest relies on code coverage feedback to guide input generation towards exploring diverse program behaviors.

In the security community, several tools have been developed to improve the effectiveness of coverage-guided fuzzing in reaching deep program states [24, 28, 50, 52, 63]. AFLGo [24] extends AFL to direct fuzzing towards generating inputs that exercise a program point of interest. It relies on call graphs obtained from whole-program static analysis, which can be difficult to compute precisely in our ecosystem. Zest is purely a dynamic analysis tool. FairFuzz [50] modifies AFL to bias input generation towards branches that are rarely executed, but does not explicitly identify parts of the program that perform the core logic. In Zest, we bias input generation towards validity no matter how frequently the semantic analysis stage is exercised.

Zest generates inputs that are syntactically valid by construction (assuming suitable generators), but uses heuristics to guide input generation towards semantic validity. Unlike Zest, symbolic execution tools [18, 21, 27, 29, 31, 39, 40, 46, 51, 65, 69] methodically explore the program under test by capturing path constraints and can directly generate inputs which satisfy specific path constraints. Symbolic execution can thus precisely produce valid inputs exercising new behavior. The cost of this precision is that it can lead to the path explosion problem for larger programs. Hybrid techniques

that combine symbolic execution with coverage-guided fuzzing have been proposed [26, 57, 68, 73]. These hybrid techniques could be combined with Zest to solve for parameter sequences that satisfy branch constraints which Zest may not cover on its own.

Grammar-based fuzzing [23, 32, 39, 56, 67] techniques rely on context-free grammar specifications to generate structured inputs. CSmith [72] generates random C programs for differential testing of C compilers. LangFuzz [43] generates random programs using a grammar and by recombining code fragments from a codebase. These approaches fall under the category of generator-based testing, but primarily focus on tuning the underlying generators rather than using code coverage feedback. Zest is not restricted to context-free grammars, and does not require any domain-specific tuning.

Several recently developed tools leverage input format specifications (either grammars [20, 71], file formats [62], or protocol buffers [66]) to improve the performance of CGF. These tools develop mutations that are specific to the input format specifications, e.g. syntax-tree mutations for grammars. Zest's generators are arbitrary programs; therefore, we perform mutations directly on the parameters that determine the execution path through the generators, rather than on a parsed representation of inputs.

There has also been some recent interest in automatically generating input grammars from existing inputs, using machine learning [41] and language inference algorithms [22]. Similarly, DIFUZE [33] infers device driver interfaces from a running kernel to bootstrap subsequent structured fuzzing. These techniques are complementary to Zest—the grammars generated by these techniques could be transformed into parametric generators for Zest.

Finally, validity feedback has been useful in fuzzing digital circuits that have constrained interfaces [48], as well as in generating seed inputs for conventional fuzzing [70].

8 CONCLUSION

We presented Zest, a technique that incorporates ideas from coverage-guided fuzzing into generator-based testing. We showed how a simple conversion of random-input generators into *parametric* input generators enables an elegant mapping from low-level mutations in the untyped parameter domain into structural mutations in the syntactically valid input domain. We then presented an algorithm that combined code coverage feedback with input-validity feedback to guide the generation of test inputs. On 5 real-world Java benchmarks, we found that Zest achieved consistently higher branch coverage and had better bug-finding ability in the semantic analysis stage than baseline techniques. Our results suggest that Zest is highly effective at testing the semantic analysis stage of programs, complementing tools such as AFL that are effective at testing the syntactic analysis stage of programs.

ACKNOWLEDGMENTS

We would like to thank Kevin Laeuffer and Michael Dennis for their helpful feedback during the development of this project, as well as Benjamin Brock and Rohan Bavishi for their invaluable comments on the paper draft. This research is supported in part by gifts from Samsung, Facebook, and Futurewei, and by NSF grants CCF-1409872 and CNS-1817122.

REFERENCES

- [1] 2018. Apache Ant. <https://ant.apache.org>. Accessed August 24, 2018.
- [2] 2018. Apache Byte Code Engineering Library. <https://commons.apache.org/proper/commons-beel>. Accessed August 24, 2018.
- [3] 2018. Apache Maven. <https://maven.apache.org>. Accessed August 24, 2018.
- [4] 2018. Google Closure. <https://developers.google.com/closure/compiler>. Accessed August 24, 2018.
- [5] 2018. Mozilla Rhino. <https://github.com/mozilla/rhino>. Accessed August 24, 2018.
- [6] 2018. ReactJS. <https://reactjs.org>. Accessed August 24, 2018.
- [7] 2019. beStorm@Software Security. <https://www.beyondsecurity.com/bestorm.html>. Accessed January 28, 2019.
- [8] 2019. Codenomicon Vulnerability Management. <http://www.codenomicon.com/index.html>. Accessed January 28, 2019.
- [9] 2019. CyberFlood - Spirent. <https://www.spirent.com/products/cyberflood>. Accessed January 28, 2019.
- [10] 2019. Eris: Porting of QuickCheck to PHP. <https://github.com/giorgiosironi/eris>. Accessed January 28, 2019.
- [11] 2019. Hypothesis for Python. <https://hypothesis.works/>. Accessed January 28, 2019.
- [12] 2019. JSVerify: Property-based testing for JavaScript. <https://github.com/jsverify/jsverify>. Accessed January 28, 2019.
- [13] 2019. PeachFuzzer. <https://www.peach.tech/>. Accessed January 28, 2019.
- [14] 2019. ScalaCheck: Property-based testing for Scala. <https://www.scalacheck.org/>. Accessed January 28, 2019.
- [15] 2019. test.check: QuickCheck for Clojure. <https://github.com/clojure/test.check>. Accessed January 28, 2019.
- [16] Cláudio Amaral, Mário Florido, and Vítor Santos Costa. 2014. PrologCheck—property-based testing in Prolog. In *International Symposium on Functional and Logic Programming*. Springer, 1–17.
- [17] Saswat Anand, Edmund K. Burke, Tsong Yueh Chen, John A. Clark, Myra B. Cohen, Wolfgang Grieskamp, Mark Harman, Mary Jean Harrold, and Phil McMinn. 2013. An orchestrated survey of methodologies for automated software test case generation. *Journal of Systems and Software* 86, 8 (2013), 1978–2001. <https://doi.org/10.1016/j.jss.2013.02.061>
- [18] Saswat Anand, Corina S. Păsăreanu, and Willem Visser. 2007. JPF-SE: a symbolic execution extension to Java PathFinder. In *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*.
- [19] Thomas Arts, John Hughes, Joakim Johansson, and Ulf T. Wiger. 2006. Testing telecoms software with quviq QuickCheck. In *Proceedings of the 2006 ACM SIGPLAN Workshop on Erlang, Portland, Oregon, USA, September 16, 2006*. 2–10. <https://doi.org/10.1145/1159789.1159792>
- [20] Cornelius Aschermann, Tommaso Frassetto, Thorsten Holz, Patrick Jauernig, Ahmad-Reza Sadeghi, and Daniel Teuchert. 2019. Nautilus: Fishing for Deep Bugs with Grammars. In *26th Annual Network and Distributed System Security Symposium (NDSS '19)*.
- [21] Thanassis Avgerinos, Alexandre Rebert, Sang Kil Cha, and David Brumley. 2014. Enhancing Symbolic Execution with Veriteesting. In *Proceedings of the 36th International Conference on Software Engineering (ICSE 2014)*. ACM, New York, NY, USA, 1083–1094.
- [22] Osbert Bastani, Rahul Sharma, Alex Aiken, and Percy Liang. 2017. Synthesizing Program Input Grammars. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)*. ACM, New York, NY, USA, 95–110. <https://doi.org/10.1145/3062341.3062349>
- [23] Michael Beyene and James H. Andrews. 2012. Generating String Test Data for Code Coverage. In *Fifth IEEE International Conference on Software Testing, Verification and Validation, ICST 2012, Montreal, QC, Canada, April 17-21, 2012*. 270–279. <https://doi.org/10.1109/ICST.2012.107>
- [24] Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed Greybox Fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*.
- [25] Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. 2016. Coverage-based Greybox Fuzzing As Markov Chain. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*.
- [26] Konstantin Böttinger and Claudia Eckert. 2016. DeepFuzz: Triggering Vulnerabilities Deeply Hidden in Binaries. In *Proceedings of the 13th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment - Volume 9721 (DIMVA 2016)*. Springer-Verlag, Berlin, Heidelberg, 25–34. https://doi.org/10.1007/978-3-319-40667-1_2
- [27] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI'08)*.
- [28] Peng Chen and Hao Chen. 2018. Angora: Efficient Fuzzing by Principled Search. In *Proceedings of the 39th IEEE Symposium on Security and Privacy*.
- [29] Vitaly Chipounov, Volodymyr Kuznetsov, and George Candea. 2012. The SZE Platform: Design, Implementation, and Applications. *ACM Transactions on Computer Systems* 30, 1 (2012), 2.
- [30] Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP)*.
- [31] Lori A. Clarke. 1976. A program testing system. In *Proc. of the 1976 annual conference*. 488–491.
- [32] David Coppit and Jiexin Lian. 2005. Yagg: An Easy-to-use Generator for Structured Test Inputs. In *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering (ASE '05)*. ACM, New York, NY, USA, 356–359. <https://doi.org/10.1145/1101908.1101969>
- [33] Jake Corina, Aravind Machiry, Christopher Salls, Yan Shoshitaishvili, Shuang Hao, Christopher Kruegel, and Giovanni Vigna. 2017. DIFUZE: Interface Aware Fuzzing for Kernel Drivers. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. ACM, New York, NY, USA, 2123–2138. <https://doi.org/10.1145/3133956.3134069>
- [34] Roy Emek, Itai Jaeger, Yehuda Naveh, Gadi Bergman, Guy Aloni, Yoav Katz, Monica Farkash, Igor Dozoretz, and Alex Goldin. 2002. X-Gen: A random test-case generator for systems and SoCs. In *High-Level Design Validation and Test Workshop, 2002. Seventh IEEE International*. IEEE, 145–150.
- [35] Robert Feldt and Simon Poulding. 2013. Finding test data with specific properties via metaheuristic search. In *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 350–359.
- [36] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: Automatic Test Suite Generation for Object-oriented Software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE '11)*.
- [37] Gordon Fraser and Andrea Arcuri. 2014. A Large-Scale Evaluation of Automated Unit Test Generation Using EvoSuite. *ACM Trans. Softw. Eng. Methodol.* 24, 2, Article 8 (Dec. 2014), 42 pages. <https://doi.org/10.1145/2685612>
- [38] Milos Gligoric, Tihomir Gvero, Vilas Jagannath, Sarfraz Khurshid, Viktor Kuncak, and Darko Marinov. 2010. Test generation through programming in UDITA. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering - Volume 1, ICSE 2010, Cape Town, South Africa, 1-8 May 2010*. 225–234. <https://doi.org/10.1145/1806799.1806835>
- [39] Patrice Godefroid, Adam Kiezun, and Michael Y. Levin. 2008. Grammar-based Whitebox Fuzzing. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '08)*.
- [40] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: Directed Automated Random Testing. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '05)*.
- [41] Patrice Godefroid, Hila Peleg, and Rishabh Singh. 2017. Learn & Fuzz: Machine Learning for Input Fuzzing. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE 2017)*. IEEE Press, Piscataway, NJ, USA, 50–59. <http://dl.acm.org/citation.cfm?id=3155562.3155573>
- [42] Marc R Hoffmann, B Janiczak, and E Mandrikov. 2011. Eclemma-jacoco java code coverage library.
- [43] Christian Holler, Kim Herzig, and Andreas Zeller. 2012. Fuzzing with Code Fragments. In *Presented as part of the 21st USENIX Security Symposium (USENIX Security 12)*.
- [44] Paul Holser. 2014. junit-quickcheck: Property-based testing, JUnit-style. <https://pholser.github.io/junit-quickcheck>. Accessed January 11, 2019.
- [45] LLVM Compiler Infrastructure. 2016. libFuzzer. <https://llvm.org/docs/LibFuzzer.html>. Accessed April 17, 2019.
- [46] James C. King. 1976. Symbolic execution and program testing. *Commun. ACM* 19 (July 1976), 385–394. Issue 7.
- [47] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating Fuzz Testing. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18)*. ACM, New York, NY, USA, 2123–2138. <https://doi.org/10.1145/3243734.3243804>
- [48] Kevin Laeuffer, Jack Koenig, Donggyu Kim, Jonathan Bachrach, and Koushik Sen. 2018. RFUZZ: Coverage-directed Fuzz Testing of RTL on FPGAs. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD '18)*. ACM, New York, NY, USA, Article 28, 8 pages. <https://doi.org/10.1145/3240765.3240842>
- [49] Leonidas Lampropoulos and Konstantinos Sagonas. 2012. Automatic WSDL-guided Test Case Generation for PropEr Testing of Web Services. In *Proceedings 8th International Workshop on Automated Specification and Verification of Web Systems, WWV 2012, Stockholm, Sweden, 16th July 2012*. 3–16. <https://doi.org/10.4204/EPTCS.98.3>
- [50] Caroline Lemieux and Koushik Sen. 2018. FairFuzz: A Targeted Mutation Strategy for Increasing Greybox Fuzz Testing Coverage. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE '18)*.
- [51] Guodong Li, Indradeep Ghosh, and Sreeranga P. Rajan. 2011. KLOVER: A Symbolic Execution and Automatic Test Generation Tool for C++ Programs. In *CAV*. 609–615.
- [52] Yuekang Li, Bihuan Chen, Mahinthan Chandramohan, Shang-Wei Lin, Yang Liu, and Alwen Tiu. 2017. Steelix: Program-state Based Binary Fuzzing. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2017)*.

- [53] Tim Lindholm, Frank Yellin, Gilad Bracha, and Alex Buckley. 2014. *The Java Virtual Machine Specification, Java SE 8 Edition* (1st ed.). Addison-Wesley Professional.
- [54] Andreas Löscher and Konstantinos Sagonas. 2017. Targeted Property-based Testing. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2017)*. ACM, New York, NY, USA, 46–56. <https://doi.org/10.1145/3092703.3092711>
- [55] A. Loscher and K. Sagonas. 2018. Automating Targeted Property-Based Testing. In *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*, Vol. 00. 70–80. <https://doi.org/10.1109/ICST.2018.00017>
- [56] Peter M. Maurer. 1990. Generating test data with enhanced context-free grammars. *Ieee Software* 7, 4 (1990), 50–55.
- [57] Saahil Ognawala, Thomas Hutzelmann, Eirini Psallida, and Alexander Pretschner. 2018. Improving Function Coverage with Munch: A Hybrid Fuzzing and Directed Symbolic Execution Approach. In *Proceedings of the 33rd Annual ACM Symposium on Applied Computing (SAC '18)*. ACM, New York, NY, USA, 1475–1482. <https://doi.org/10.1145/3167132.3167289>
- [58] OW2 Consortium. 2018. ObjectWeb ASM. <https://asm.ow2.io>. Accessed August 21, 2018.
- [59] Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball. 2007. Feedback-Directed Random Test Generation. In *Proceedings of the 29th International Conference on Software Engineering (ICSE '07)*. IEEE Computer Society, Washington, DC, USA, 75–84. <https://doi.org/10.1109/ICSE.2007.37>
- [60] Rohan Padhye, Caroline Lemieux, and Koushik Sen. 2019. JQF: Coverage-guided Property-based Testing in Java. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '19)*. <https://doi.org/10.1145/3293882.3339002>
- [61] Manolis Papadakis and Konstantinos Sagonas. 2011. A PropEr Integration of Types and Function Specifications with Property-based Testing. In *Proceedings of the 10th ACM SIGPLAN Workshop on Erlang (Erlang '11)*. ACM, New York, NY, USA, 39–50. <https://doi.org/10.1145/2034654.2034663>
- [62] Van-Thuan Pham, Marcel Böhme, Andrew E. Santosa, Alexandru Razvan Caciulescu, and Abhik Roychoudhury. 2018. Smart Greybox Fuzzing. *CoRR* abs/1811.09447 (2018). [arXiv:1811.09447](http://arxiv.org/abs/1811.09447) <http://arxiv.org/abs/1811.09447>
- [63] Sanjay Rawat, Vivek Jain, Ashish Kumar, Lucian Cojocar, Cristiano Giuffrida, and Herbert Bos. 2017. VUzzer: Application-aware Evolutionary Fuzzing. In *Proceedings of the 2017 Network and Distributed System Security Symposium (NDSS '17)*.
- [64] Alexandre Rebert, Sang Kil Cha, Thanassis Avgerinos, Jonathan Foote, David Warren, Gustavo Grieco, and David Brumley. 2014. Optimizing Seed Selection for Fuzzing. In *Proceedings of the 23rd USENIX Conference on Security Symposium (SEC'14)*. USENIX Association, Berkeley, CA, USA, 861–875. <http://dl.acm.org/citation.cfm?id=2671225.2671280>
- [65] Koushik Sen, Darko Marinov, and Gul Agha. 2005. CUTE: A Concolic Unit Testing Engine for C. In *Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE-13)*.
- [66] Kostya Serebryany, Vitaly Buka, and Matt Morehouse. 2017. Structure-aware fuzzing for Clang and LLVM with libprotobuf-mutator.
- [67] Emin Gün Sirer and Brian N. Bershad. 1999. Using Production Grammars in Software Testing. In *Proceedings of the 2Nd Conference on Domain-specific Languages (DSL '99)*. ACM, New York, NY, USA, 1–13. <https://doi.org/10.1145/331960.331965>
- [68] Nick Stephens, John Grosen, Christopher Salls, Andrew Dutcher, Ruoyu Wang, Jacopo Corbetta, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. 2016. Driller: Augmenting Fuzzing Through Selective Symbolic Execution. In *Proceedings of the 2016 Network and Distributed System Security Symposium (NDSS '16)*.
- [69] Nikolai Tillmann and Jonathan de Halleux. 2008. Pex - White Box Test Generation for .NET. In *Proceedings of Tests and Proofs*.
- [70] J. Wang, B. Chen, L. Wei, and Y. Liu. 2017. Skyfire: Data-Driven Seed Generation for Fuzzing. In *2017 IEEE Symposium on Security and Privacy (SP) (SP '17)*.
- [71] Junjie Wang, Bihuan Chen, Lei Wei, and Yang Liu. 2019. Superion: Grammar-Aware Greybox Fuzzing. In *41st International Conference on Software Engineering (ICSE '19)*.
- [72] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '11)*.
- [73] Insu Yun, Sangho Lee, Meng Xu, Yeongjin Jang, and Taesoo Kim. 2018. QSYM: A Practical Concolic Execution Engine Tailored for Hybrid Fuzzing. In *Proceedings of the 27th USENIX Conference on Security Symposium (SEC'18)*. USENIX Association, Berkeley, CA, USA, 745–761. <http://dl.acm.org/citation.cfm?id=3277203.3277260>
- [74] Michał Zalewski. 2014. American Fuzzy Lop. <http://lcamtuf.coredump.cx/afl>. Accessed January 11, 2019.
- [75] Michał Zalewski. 2016. FidgetyAFL. <https://groups.google.com/d/msg/afl-users/foPeb62FZUG/CES5lhznDgAJ>. Accessed Jan 28th, 2019.